Advanced Programming Practice · 21CSC203P

Unit 1 · Session 8 · SLO 1 — Solved Worksheet

Name: DEEP SAND DEEP SAND | **Reg. No:** RA2511026010557

Unit – 1 : Session – 8 : SLO – 1 | Understand Concurrency

Activity (Solutions)

• 1) A thread is the smallest unit of execution within a program — a lightweight path through code. A process is an independent program instance with its own memory space; threads share the process's memory (heap) but have their own stack and program counter, making inter-thread communication cheap but inter-process communication expensive.

• 2) Concurrency issues in shared memory: race conditions (two threads updating a balance simultaneously → lost updates), deadlocks (threads holding locks and waiting on each other), starvation, and visibility problems (stale cached values) — fixed with synchronized blocks, locks, and volatile variables.

• 3) Thread lifecycle: NEW (created, not started) → RUNNABLE (start() called, ready/running) → BLOCKED/WAITING/TIMED_WAITING (waiting for a lock, wait(), or sleep(ms)) → TERMINATED (run() completes). Example: Thread t = new Thread(() -> { try{ Thread.sleep(100);}catch(Exception e){} System.out.println("done"); }); t.start(); — t moves NEW → RUNNABLE → TIMED_WAITING → TERMINATED.